

Лабораторная работа №2

Шифры перестановки: маршрутное шифрование, шифрование с помощью решёток, таблица Виженера

Кюнкриков Даниил Саналович

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
3.1	Маршрутное (колоночное) шифрование	7
3.2	Шифрование с помощью решёток (решётка Кардано)	7
3.3	Таблица Виженера	8
4	Выполнение лабораторной работы	9
4.1	Маршрутное шифрование	9
4.2	Шифрование с помощью решёток	11
4.3	Таблица Виженера	13
5	Результаты работы	16
5.1	Маршрутное шифрование — пример	16
5.2	Решётка 4×4 — пример	17
5.3	Виженер — пример	17
6	Выводы	19

Список иллюстраций

5.1	Результат работы функции маршрутного шифрования	16
5.2	Результат работы функции шифрования по решётке 4×4	17
5.3	Результат работы функции по таблице Виженера	18

Список таблиц

1 Цель работы

Изучить принципы работы шифров перестановки и реализовать на языке **Julia** три алгоритма:

- маршрутное (колоночное) шифрование по ключевому слову;
- шифрование с помощью решёток (решётка 4×4);
- шифрование по таблице Виженера.

2 Задание

1. Реализовать указанные алгоритмы в режимах шифрования и расшифровки (через меню программы).
2. Провести проверку работы программ на тестовых сообщениях и ключах.
3. Оформить отчёт о проделанной работе.

3 Теоретическое введение

Шифры перестановки изменяют **порядок** символов открытого текста без замены самих символов. Такие методы относятся к классической криптографии и часто рассматриваются в историческом контексте вместе с простыми подстановками [2]. Несмотря на простоту, табличные перестановки удобно использовать для объяснения ключевых идей: представление текста в матрице, применение перестановки по ключу и восстановление исходного порядка.

3.1 Маршрутное (колоночное) шифрование

Текст записывается в таблицу $m \times n$, где n — длина ключевого слова. Далее столбцы упорядочиваются по алфавиту букв ключа (формируется перестановка столбцов), и таблица считывается по столбцам в новом порядке. Для расшифрования требуется построить обратную перестановку и восстановить исходное размещение символов.

3.2 Шифрование с помощью решёток (решётка Кардано)

Решётка Кардано — это трафарет (маска) с «прорезями», через которые последовательно записываются буквы текста. После поворотов (обычно на 90°) маска открывает новые позиции, пока не будет заполнена вся таблица [2].

Ключом служит расположение прорезей (и порядок поворотов).

3.3 Таблица Виженера

Шифр Виженера относится к полиалфавитным подстановкам: сдвиг определяется символом ключа, который повторяется до длины сообщения. В терминах индексов алфавита длины n :

- **шифрование:** $C_i = (P_i + K_i) \bmod n$;
- **расшифровка:** $P_i = (C_i - K_i) \bmod n$,

где K_i — индекс i -го символа повторяющегося ключа [1; 2].

4 Выполнение лабораторной работы

Ниже приведены листинги программных реализаций (Julia).

4.1 Маршрутное шифрование

Программная реализация маршрутного шифрования показана на Листинге №1.

4.1.1 Листинг №1 — Маршрутное шифрование (колоночная перестановка)

```
function marshr_main()
    # Бесконечный цикл для работы программы до команды выхода

    while true
        # Выводим меню с доступными командами
        println("ш - шифрование, р - расшифровка, в - выход")
        #menu = lowercase(strip(readline()))
        cmd = lowercase(strip(readline()))
        cmd == "в" && (println("выход");
            break)
        cmd in ["ш","р"] || (println("Ошибка команды"); continue)
```

```

# Запрашиваем текст для шифрования/расшифрования и пароля шифра
print("Введите сообщение:")
text = readline()
print("Введите пароль :")
password = readline()

# подготовка текста
# удаление пробелов и приводим к верхнему регистру
clean_text = replace(uppercase(text), " " => "")
# преобразуем пароль в массив символов (для избежания проблем с индексацией р
pass_chars = collect(uppercase(password))
# n - количество столбцов (равно длине пароля)
# m - количество строк
n, m = length(pass_chars), ceil(Int, length(clean_text) / length(pass_chars))

padded = clean_text * "A"^(m*n - length(clean_text))
table = reshape(collect(padded), (m, n))
# создание таблицы
column_pairs = [(pass_chars[i], i) for i in 1:length(pass_chars)]
# Сортируем пары по символам пароля (алфавитный порядок)
sort!(column_pairs, by = x -> x[1])
sorted_cols = [idx for (char, idx) in column_pairs]
result = join([table[i,j] for j in sorted_cols for i in 1:m])
# вывод результата
println("Result: $result")

end
end

```

```
marshr_main()
```

4.2 Шифрование с помощью решёток

Программная реализация шифрования с помощью решёток показана на Листинге №2.

4.2.1 Листинг №2 — Шифрование с помощью решёток

```
function cellbased_main()
    while true
        println("ш - шифрование, р - расшифровка, в - выход")

        cmd = lowercase(strip(readline()))
        cmd == "в" && (println("Выход");
        break)

        cmd in ["ш","р"] || (println("Ошибка команды"); continue)
        print("Введите сообщение:")
        # Запрашиваем текст для шифрования/расшифрования и пароля шифра

        text = readline()
        # должен содержать 4 символа для решетки 2x2
        print("Введите пароль (4 символа):")
        password = readline()

        clean_chars = collect(replace(uppercase(text), " " => ""))
```

```

pass_chars = collect(uppercase(password))

k = 2 # размер решетки
# Размер большой решетки (2k × 2k = 4x4)
size_2k = 2*k
# Создаем булеву маску (false - закрыто, true - прорезь)
grille = falses(size_2k, size_2k)
# Заполняем маску прорезями в 4 угловых квадратах 2x2
for i in 1:k, j in 1:k
    grille[i,j] = grille[i,k+j] = grille[k+i,j] = grille[k+i,k+j] = true
end

total = size_2k^2

length(clean_chars) < total && append!(clean_chars, fill('A', total - length(
table, idx, mask = fill(' ', size_2k, size_2k), 1, copy(grille)

for _ in 1:4
    for i in 1:size_2k, j in 1:size_2k
        mask[i,j] && idx <= length(clean_chars) && (table[i,j] = clean_chars[
    end
    mask = reverse(mask,dims=1)
end

sorted_cols = sort(1:length(pass_chars), by = i ->pass_chars[i])
result = join([table[i,j] for j in sorted_cols for i in 1:size_2k])

println("Result: $result")

```

```
    end
end

cellbased_main()
```

4.3 Таблица Виженера

Программная реализация таблицы Виженера показана на Листинге №3.

4.3.1 Листинг №3 — Таблица Виженера

```
function vigener()
    alphabet = collect("АБВГДЕЁЖИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ")
    n = length(alphabet)

    while true
        println("ш - шифрование, р - расшифровка, в - выход")

        cmd = lowercase(strip(readline()))
        cmd == "в" && (println("выход"); break)

        cmd in ["ш","р"] || (println("Ошибка команды"); continue)
        print("Введите сообщение:")
        text = readline()
        print("Введите пароль :")
        password = readline()
```

```

clean_chars = collect(replace(uppercase(text), " " => ""))
pass_chars = collect(replace(uppercase(password), " " => ""))

# Создаем пустой массив символов для ключа
key_chars = Char[]

# Для каждого символа текста определяем соответствующий символ ключа
for i in 1:length(clean_chars)
    push!(key_chars, pass_chars[(i-1) % length(pass_chars) + 1])
end

result_chars = Char[]
# Обрабатываем каждый символ текста
for i in 1:length(clean_chars)
    text_char = clean_chars[i]
    key_char = key_chars[i]

    # Находим позиции символов в алфавите
    text_idx = findfirst(==(text_char), alphabet)
    key_idx = findfirst(==(key_char), alphabet)

    if text_idx != nothing && key_idx != nothing
        if cmd == "Ш"
            new_idx = (text_idx + key_idx - 1) % n
            new_idx == 0 && (new_idx = n)
        else
            new_idx = (text_idx - key_idx) % n
            new_idx == 0 && (new_idx = n)
        end
    end

    # Добавляем преобразованный символ к результату

```

```
        push!(result_chars, alphabet[new_idx])
    else
        push!(result_chars, text_char)
    end
end
result = String(result_chars)

println("Result: $result")
end
end

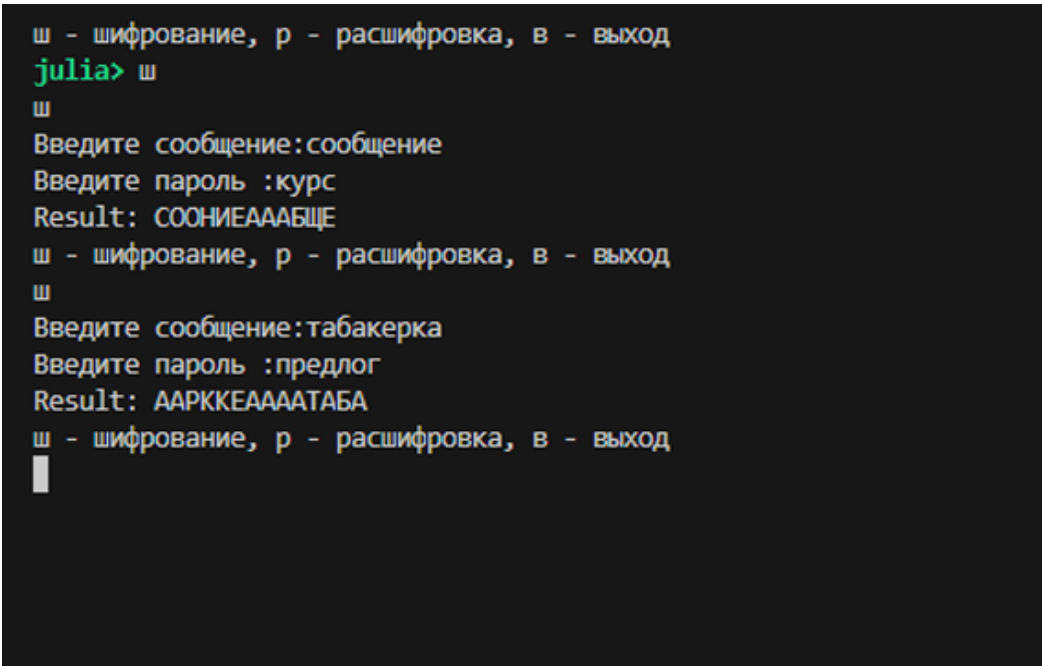
vigenere()
```

5 Результаты работы

В качестве проверки были выполнены тестовые прогоны алгоритмов (пример ввода/вывода).

5.1 Маршрутное шифрование — пример

Результат программной реализации маршрутного шифрования представлен на рисунке Рисунок 5.1.



```
ш - шифрование, р - расшифровка, в - выход
julia> ш
ш
Введите сообщение:сообщение
Введите пароль :курс
Result: COONIEAAAБЩЕ
ш - шифрование, р - расшифровка, в - выход
ш
Введите сообщение:табакерка
Введите пароль :предлог
Result: AАРККЕАААТАБА
ш - шифрование, р - расшифровка, в - выход
█
```

Рисунок 5.1: Результат работы функции маршрутного шифрования

5.2 Решётка 4×4 — пример

Результат программной реализации шифрования с помощью решёток представлен на рисунке Рисунок 5.2.

```
ш - шифрование, р - расшифровка, в - выход
julia> ш
ш
Введите сообщение:сообщение
Введите пароль :курс
Result: СЩЕАОНААБИААОЕАА
ш - шифрование, р - расшифровка, в - выход
ш
Введите сообщение:табакерка
Введите пароль :стол
Result: АКААБРААТКАААЕАА
ш - шифрование, р - расшифровка, в - выход
```

Рисунок 5.2: Результат работы функции шифрования по решётке 4×4

5.3 Вижнер — пример

Результат программной реализации таблицы Вижнера представлен на рисунке Рисунок 5.3.

```
ш - шифрование, р - расшифровка, в - выход
julia> ш
ш
Введите сообщение:сообщение
Введите пароль :пароль
Result: БОЯПЕБЭИХ
ш - шифрование, р - расшифровка, в - выход
ш
Введите сообщение:табакерка
Введите пароль :курс
Result: ЭУССХШБЬК
ш - шифрование, р - расшифровка, в - выход
█
```

Рисунок 5.3: Результат работы функции по таблице Виженера

6 Выводы

1. Реализованы три криптографических алгоритма: маршрутное шифрование, шифрование с помощью решёток и таблица Виженера.
2. Реализован консольный интерфейс с выбором режима (шифрование/расшифровка) и вводом ключа.
3. Выполнено тестирование на примерах, подтверждающее работоспособность реализованных программ.

Список литературы

1. *Menezes A. J., van Oorschot P. C., Vanstone S. A.* Handbook of Applied Cryptography. — CRC Press, 1996. — ISBN 0-8493-8523-7.
2. *Singh S.* The Code Book: The Science of Secrecy from Ancient Egypt to Quantum Cryptography. — Fourth Estate, 1999. — ISBN 1857028799.